Fondamenti di Cybersecurity
Appunti

Giovanni "Qua' Qua' dancer" Palma
e
Alex "Morbidelli WhatsApp" Basta

Contents

Chapter 1 **Key Exchange** Page

1.1 Introduction to Cryptography
Encryption Terminology — • Goals and Protocols — • Kerchoff's Principle and the Threat Model — • Symmetric Encryption — • Asymmetric Encryption — • Secure Communication — • Boh — • Stream cipher — • One-Time Pad — • Pseudo - Random Generators e Stream Ciphers —

1.2 Block Ciphers
DES — • Double DES — • Triple DES — • DESX — • Attacks on block ciphers — • AES —

Chapter 2 **Modular Arithmetic** Page

Chapter 3 **Asymmetric Criptography** Page

Chapter 4 **IPsec and TLS** Page

Chapter 5 **Access Control** Page

Chapter 6 **Exploits and Patches** Page

Ci sara' una domanda sul lab

Example 0.0.1

Quali opzioni ho per crackare una password?

Chapter 1

Key Exchange

1.1 Introduction to Cryptography

Definition 1.1.1: Cryptography

Art and science of using mathematics to obscure the meaning of data by applying transformations to the data thta are impractical or impossible to reverse without the knowledge of some key

Definition 1.1.2: Cryptoanalysis

Art/science of breaking encryption without knowing the key

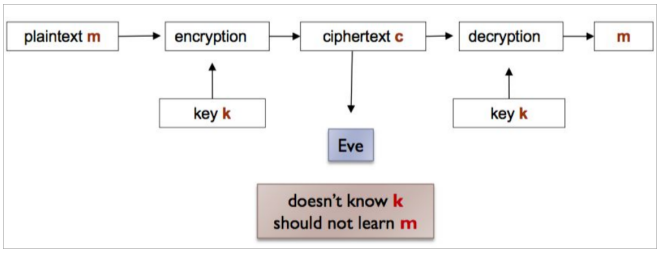
Used for:

- **Communication**: web traffic, wireless, vpn
- **Files on disk**
- **User authentication**

For secure communication, we also want to ensure no eavesdropping or tampering. Possible approaches are:

- **Steganography**: we 'hide' the existance of the message
- **Cryptography**: we instead hide the meaning of the message

1.1.1 Encryption Terminology



1.1.2 Goals and Protocols

The basic goals are:

- **Privacy**
- **Authenticity**

- **Integrity**
- **Non-repudiation**: no disclaming of authorship (guarantees Authenticity and Integrity)

The *protocols* need to guarantee these goals by understanding:

- The parties and the context
- The goals
- The **trusted computing base**
- The capabilities of the ... (**Threat Model**)

1.1.3 Kerchoff's Principle and the Threat Model

Important rule regarding the safety of cyber systems

> **Theorem 1.1.1**
> The security of a protocol shouldn't assume that the underlying methods/algorithms of the encryption are secret, as only the secrecy of the keys can be guaranteed.
> **Security by obscurity does not work.**

So the encryption functions need to remain secure even with the attacker knowing how the function works.
The attacker threat model consists of:

- Knowledge about the cipher (Kerchoff)
- Interaction with the messages and the protocol
- Interaction with the encryption algorithm
  - **Ciphertext-only**
  - **Chosen-plaintext attack (CPA)**
  - **Chosen-ciphertext attack (CCA)**
  - CPA and CCA may be *adaptive* (previous requests may change choices)
- Available resources (storage/computation)

1.1.4 Symmetric Encryption

Lowkey l'abbiamo fatta gia' due volte, aggiungo solo roba che dice in piu' (sta letteralmente leggendo le slide come il guerriero)

- **Single-use key**: the key is regenerated for each transmitted file
- **Multi-use key**: the key is used with a "nonce" in order to be used for multiple transmissions

1.1.5 Asymmetric Encryption

Private and public key. La pubblica e' salvata in una **public repo** gestita da un'autorita' trusted

1.1.6 Secure Communication

Una volta SSL ora TLS, poi RCS per end-to-end security (sono la stessa cosa??)

1.1.7 Boh

Criptography is a rigorus science that follows three main steps:

- Specify Threat Model
- Propose a construction
- ...

un po di storia, va veloce (manca la post-quantum criptology)
vabbe' sto scrivendo tutot in italiano facciamolo in italiano
la brodez sta davvero ripetendo il cifrario di Cesare.... e' ez da battere provando tutte le chiavi (non ce ne stanno molti) -¿ bruteforce/exhaustive-search criptanalysis

Monoalphabetic cipher

Come chiave prende una permutazione dell'alfabeto sono 2^{88} non possiamo fare brute force possiamo fare criptanalysis con metodi statistici, ovvero usando la frequenza di ogni lettera

Affine cipher

Caso specifico del sub-cipher1

1.1.8 Stream cipher

in Un simmetric cipher definito su (K, P, C), con K chiave, plaintext P e ciphertext C, abbiamo che la funzione di encryption è spesso randomizzata ed è definita come E : K x P -¿ C e invece la funzione di decryption è deterministica e tale per cui D: K x C -¿ P.

1.1.9 One-Time Pad

Definition 1.1.3: One-Time Pad

Si definisce *One-Time Pad* il seguente cifrario:

- Spazio delle chiavi: $0, 1^n$ (stringhe di bit di lunghezza n)
- Funzione di encryption: $E_k(m) = m \oplus k$
- Funzione di decryption: $D_k(c) = c \oplus k$

Dove k è la chiave e m è il plaintext e `len(k) = len(m)`

Praticamente si fa lo XOR tra i bit del plaintext e quelli della chiave. In questo caso la lunghezza della key deve essere uguale a quella del plaintext.

Proposition 1.1.1 Sulla corrtettezza dell'Otp

$$D(k, E(k, m)) = k \oplus (k \oplus m) = (k \oplus k) \oplus m = 0 \oplus m = m$$

pro: è mega veloce sia in encyption che in decryption brodez contro: deve usare chiavi lunghissime perché la key deve essere lunga come il messaggio. ma dato che questa key deve essere mandata da alice a bob in modo sicuro, allora potrebbe usare sto meccanismo sicuro per mandare direttamente il messagio, quindi boh non serve a un cazzo.

Sicurezza del cifrario OTP

PEr supporre che un messaggio sia sicuro, si potrebbero usare due approcci naif:

- **L'attaccante non riesce a trovare la chiave**: Non basta. Magari non trova la chiave ma riesce a decifrare il messaggio lo stesso
- **L'attaccante non riesce a recuperare l'intero plaintext**: Non basta neanche. Se recupera anche solo un bit del messaggio, per molte applicazioni è già un disastro

SI RINGRAZIA PROFONDAMENTE IL BUON CLAUDE SHANNON PER LA DEFINIZIONE DI SICUREZZA, OVVERO CHE *ciphertext non deve rivelare assolutamente nessuna informazione sul plaintext*. Zero. Nulla. Neanche quante vocali ci sono, neanche la lunghezza approssimativa. Si noti la la seguente definizione:

Definition 1.1.4: Sicurezza perfetta secondo Shannon

Un cifrario (E, D) su (K, P, C) ha *perfect secrecy* se $\forall m_0, m_1 \in P : len(m_0) = len(m_1)$ e $\forall c \in C$ allora:

$$Pr[E(k, m_0) = c] = Pr[E(k, m_1) = c]$$

Dove k è la chiave

Note:

In altre parole la probabilità che il chipertext c è idendica qualunque sia il messaggio cifrato. PErtanto non c'è informazione in c su quale sia m_0 o m_1

Si noti il seguente teorema:

Theorem 1.1.2

Il One-Time Pad ha perfect secrecy

Dimostrazione: Sia m e c, vogliamo calcolare $Pr[E(k, m) = c]$ dove k è uniforme su K.
Tale probabilità è uguale al numero di chiavi $k \in K$ tali che $E(k, m) = c$ diviso al numero di chiavi totali $|K|$, pertanto basta dimostrare che questa probabilità è costante $\forall (m, c)$
Nell'OTP $E(k, m) = m \oplus k$ implica $k = m \oplus c$. C'è esattamente una chiave per ogni coppia (m, c) che produce quel ciphertext. Il numeratore quindi è sempre 1, indipendemente da m e c.
Pertanto si che

$$Pr[E(k, m) = c] = \frac{1}{|K|} \quad \forall m, c$$

La probabilità è la stessa per qualunque messaggio

Si ha tuttavia una conseguenza importante

Theorem 1.1.3 Teorema di Shannon

La sicurezza perfetta implica che $|K| \geqslant |P|$

Note:

Ovvero un cifrario ha sicurezza perfetta se e solo se la lunghezza della chiave è almeno quella del messaggio

Se volete sicurezza perfetta, pagate il prezzo di chiavi lunghissime. E questo è impraticabile su larga scala.
La conseguenza pratica è drastica: dobbiamo rinunciare alla sicurezza perfetta e accontentarci di qualcosa di "meno perfetto", la sicurezza computazionale, che è comunque più che sufficiente.

1.1.10 Pseudo - Random Generators e Stream Ciphers

Abbiamo due fatti:

- Il OTP è perfettamente sicuro ma richiede chiavi lunghe quanto il messaggio

- La sicurezza perfetta è impossibile con chiavi corte.

La soluzione è è quella di rinunciare alla sicurezza perfetta e sostituiamo la chiave veramente veramente casuale con una sequenza pseudo-casuale generata da una chiave corta. Questo è l'idea degli stream cipher. Formalmente

Definition 1.1.5: Stram Cipher

Si definisce *Stream Cipher* un sistema che mira a emulare la sicurezza del One-Time Pad in modo efficiente. A differenza dei cifrari a blocchi che operano su segmenti fissi di dati, lo stream cipher elabora il testo in chiaro bit per bit o byte per byte

Adesso si noti la seguente definizione

Definition 1.1.6: Pseudo-Random Generator (PRG)

Si definisce *PRG* la seguente funzione:

$$G : \{0,1\}^s \to \{0,1\}^n \text{ con } n >> s$$

In prarica riceve in input un seed (la chiave breve, s bit) e produce una sequenza molto più lunga (n bit) che sembra casuale. Lo stream cipher usa G così

- $E(k, m) = G(k) \oplus m$
- $D(k, c) = G(k) \oplus c$

La chiave k è breve (il seed), la sequenza $G(k)$ è lunga quanto il messaggio.

Tre Tipi di Generatori

Vale la pena distinguere tre concetti

Definition 1.1.7: TRNG (True Random Number Generator)

usa fenomeni fisici genuinamente casuali ex rumore termico, eventi radioattivi, jitter dell'orologio. Produce vera casualità, ma è lento

Definition 1.1.8: PRNG (Pseudo-Random Number Generator)

algoritmo deterministico che a partire da un seed produce una sequenza lunga e statisticamente "casuale"

Note:
Veloce ma deterministico, da un seed produce sempre la stessa sequenza

Definition 1.1.9: PRF (Pseudo-Random Function)

una funzione che, dato un input, produce un output pseudo-casuale

Note:
è una generalizzazione del PRNG

Requisiti di un buon PRNG

Un PRNG per uso crittografico deve soddisfare due requisiti fondamentali

- **Randomness (apparente)**: la sequenza prodotta deve superare tutti i test statistici noti — distribuzione uniforme dei bit, assenza di pattern, correlazioni basse tra bit vicini
- **Unpredictability (imprevedibilità)**: dato un prefisso della sequenza, non deve esistere nessun algoritmo efficiente (in tempo polinomiale) in grado di predire il bit successivo con probabilità significativamente maggiore di 1/2 (**next-bit test**)

PRNG deboli, NON USARE @ALEXBASTA, NON USARLI PERFAVORE

Due esempi di PRNG statisticamente accettabili ma crittograficamente insicuri

- `glibc random()`: Utilizza un generatore a feedback lineare (non-linear additive feedback). conoscendo alcuni output si può calcolare facilmente il seed e predire tutti gli output futuri
- **Linear Congruential Generator**: Si ottiene attravero questo algoritmo `r[i] = (a · r[i-1] + b) mod p`. Semplice, veloce, ha buone proprietà statistiche, ma è matematicamente elementare da invertire: dati due output consecutivi, si trovano immediatamente a e b

Un Buon PRNG: Blum Blum Shub (BBS)

Il BBS è un esempio di PRNG crittograficamente sicuro la cui sicurezza è ridotta a un problema matematico difficile

Algorithm 1: Generatore Blum Blum Shub

```
Scegli due grandi numeri primi p e q tali che p ≡ q ≡ 3 (mod 4);
Calcola n ← p · q;
Scegli un seed s tale che gcd(s, n) = 1;
Calcola X_0 ← s^2 mod n;
for i = 1 to k do
    X_i ← X_{i-1}^2 mod n;
    B_i ← X_i mod 2;
return {B_1, B_2, ..., B_k};
```

Proposition 1.1.2 Sicurezza di BBS

Predire il bit successivo della sequenza BBS è computazionalmente equivalente a fattorizzare $n = p \cdot q$

Note:

non esiste nessun algoritmo efficiente noto per fattorizzare numeri grandi pertanto BBS è crittograficamente sicuro

Tuttavia è relativamente lento rispetto ad altri PRNG crittografici. Per applicazioni ad alta velocità si usano altri approcci.

Formalizzare queste note TODO

Gli Stream Cipher: Struttura Generica Uno stream cipher tipico funziona così: a partire dalla chiave K e da un valore di inizializzazione IV, si mantiene uno stato interno s_i che evolve a ogni passo. A ogni passo si produce un byte del keystream z_i tramite una funzione $g(s_i)$, e lo stato avanza con una funzione $f(s_i)$. Il keystream viene poi XOR-ato byte a byte con il plaintext: $c_i = p_i \oplus z_i$. L'IV serve a garantire che lo stesso key K produca keystream diversi in sessioni diverse — un meccanismo simile al nonce che vedremo nei cifrari moderni.
Gli Stream Cipher Non Hanno Perfect Secrecy Una domanda naturale: uno stream cipher può avere perfect secrecy? No, per definizione. La chiave k è corta (il seed), mentre il messaggio è lungo. Per il teorema di Shannon, $|K| \geqslant |P|$ è necessario per la perfect secrecy — condizione violata da qualsiasi stream cipher pratico. Dobbiamo quindi usare una definizione diversa di sicurezza, la sicurezza computazionale: il sistema è sicuro se non esiste nessun algoritmo efficiente in grado di distinguere il keystream G(k) da una sequenza veramente casuale. La sicurezza dello stream cipher dipende interamente dalla qualità del PRG: un PRG computazionalmente indistinguibile dal random implica uno stream cipher computazionalmente sicuro.

RC4

Il RC4 (Rivest Cipher 4) è stato lo stream cipher dominante per decenni, usato in SSL/TLS per HTTPS e in WEP/WPA per le reti Wi-Fi. Oggi, tuttavia, è ufficialmente deprecato ed è considerato insicuro per qualsiasi applicazione crittografica.

Struttura : usa un array di stato S di 256 byte (una permutazione dei valori 0-255) e una chiave di lunghezza variabile tra 8 e 2048 bit

Fase 1 :

Algorithm 2: Inizializzazione di S

```
for i = 0 to 255 do
    S[i] = i;
    T[i] = K[i mod keylen];
j = 0;
for i = 0 to 255 do
    j = (j + S[i] + T[i]) mod 256;
    swap(S[i], S[j]);
```

Note:
Inizializza l'array di 256 byte. Prende la tua chiave segreta (che può variare da 40 a 2048 bit) e la utilizza per "mescolare" l'array

Fase 2 :

Algorithm 3: Generazione del keystream

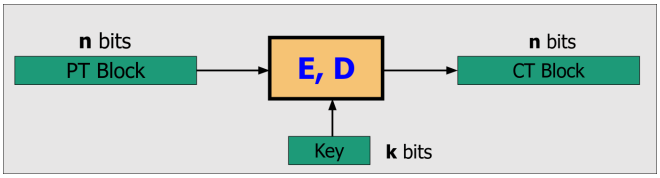
```
i = 0;
j = 0;
while true do
    i = (i + 1) mod 256;
    j = (j + S[i]) mod 256;
    swap(S[i], S[j]);
    t = (S[i] + S[j]) mod 256;
    output S[t];
```

Note:
A ogni ciclo, continua a scambiare due byte all'interno dell'array ed estrae un valore pseudo-casuale in uscita

1.2 Block Ciphers

Lavora su blocchi di lunghezza fissa, puo' generare un po' di delay. I principali sono:

- **DES**: $n = 64$ bits, $k = 56$ bits (ma possiamo anche dire 64, vedremo perche')
- **3DES**: $n = 64$ bits, $k = 3 \cdot 56 = 168$ bits
- **AES**: $n = 128$ bits, $k = 56$ bits



Il block cipher e' costruito per iterazioni:

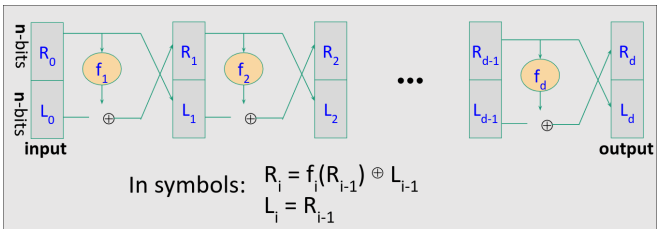
1. Key expansion: partendo da chiave lunga, genero l sottochiavi k_i (dove l e' il numero di iterazioni)

2. Applico la **round function** a m_i e la sottochiave k_i per generare m_{i+1} (dove m_1 e' il plain text)

Il problema e' che e' molto lento rispetto agli altri cifrari, cosa che AES migliora abbastanza rispetto a DES.

1.2.1 DES

L'idea core e' il *Feistel Network*:
Given functions $f_1, ..., f_d : \{0,1\}^n \to \{0,1\}^n$ (non per forza invertibili) dobbiamo costruire una funzione **invertibile** $F : \{0,1\}^{2n} \to \{0,1\}^{2n}$



Come possiamo calcolare $R_i = L_{i-1} + f_i(R_{i-1})$ e $L_i = R_{i-1}$, allora:

> **Proposition 1.2.1**
> Per ogni $f_1, ..., f_d$, il Feistel network is **invertible**.

Dimostrazione per Costruzione: Boh lo dimostra, guarda il libr onon ssi capisce niente.

Per fare la decription, basta applicare lo stesso circuito in ordine opposto, WOW!
La lunghezza della chiave di DES in verita' e' composta da 8 byte, ma l'ottavo bit di ognuno viene usato come parity-check

1.2.2 Double DES

Definiamo una funzione $2E : K^2 \times M \to M$ come $2E(k_1, k_2, m) = E(k_1, E(k_2, m))$, quindi la lunghezza della chiave viene raddoppiata $2 * 56 = 112$.
Per la decription applichiamo le chiavi in ordine inverso:

$$D(k_2, D(k_1, m))$$

Ma c'e' un problema: dato m e c, lo scopo e' di trovare (k_1, k_2) t.c.:

- $E(k_1, E(k_2, m))$
- $E(k_2, m) = D(k_1, c)$

Che sono equivalenti. Questo ci da' l'apertura per un attacco a due parti:

1. Creo una tablella che per ogni riga ha la tupla $(k_i, E(k_i, m))$ e la ordino per k
2. Per ogni k, testa se $D(k, c)$ sta nella seconda colonna. Se c'e', allora

$$E(k_i, m) = D(k, c) \implies (k_i, k) = (k_2, k_1)$$

Il costo computazionale e' di

$$2^{56} log(2^{56}) + 2^{56} log(2^{56}) < 2^{63} \ll 2^{112}$$

Dato che dobbiamo costruire e ordinare la tabella, poi cercare all'interno della tabella. DA STUDIARE LA METTE SEMPRE STA ROBA IN ESAME!!!!

1.2.3 Triple DES

Stessa roba del double, abbiamo tre chiavi, pero' e' un po' diverso

$$3E(k_1, k_2, k_3, m) = E(k_1, D(k_2, E(k_3, m)))$$

E' stra lento (3 operaizoni). Se $k_1 = k_2 = k_3$ ci troviamo nel caso di DES.
Ci sta sempre il meet in the middle attack, quindi il costo e' di $< 2^{118}$ (comunque non poco).

1.2.4 DESX

Definiamo

$$EX(k_1, k_2, k_3, m) = k_1 \oplus E(k_2, m \oplus k_3)$$

Facile da attaccare in tempo 2^{120}

1.2.5 Attacks on block ciphers

Differential e' difficile su DES, ma si puo' usare il linear cryptanalysis dato che e' una cifratura iterativa. o

1.2.6 AES

DES non e' troppo efficente

Chapter 2

Modular Arithmetic

Chapter 3

Asymmetric Criptography

Chapter 4

IPsec and TLS

Chapter 5

Access Control

Chapter 6

Exploits and Patches

Chapter 7

Wireless Network Security

7.1 L'ambiente wireless/radio

Per capire la sicurezza delle reti wireless bisogna prima capire l'ambiente in cui operano. Una rete wireless è costituita da **tre componenti** fondamentali, ognuno dei quali rappresenta un potenziale punto di attacco:

- **Client wireless**: il dispositivo dell'utente finale, come telefono, laptop, tablet, sensore IoT, dispositivo Bluetooth
- **Access Point (AP)**: l'infrastruttura che fornisce connettività (base station cellulare, hotspot Wi-Fi, router domestico, ecc.)
- **Mezzo trasmissivo**: l'aria stessa, ovvero le onde radio che trasportano i dati tra client e AP

7.1.1 Perché il wireless è più rischioso del cablato

Ci sono quattro ragioni strutturali per cui le reti wireless sono intrinsecamente più vulnerabili:

- **Canale broadcast**: nelle reti cablate un segnale viaggia su un cavo dedicato, e per intercettarlo serve accesso fisico. In una rete wireless il segnale si propaga in tutte le direzioni: chiunque sia nel raggio può ascoltarlo. L'*eavesdropping* diventa banale.
- **Mobilità**: i dispositivi cambiano rete, si riautenticano, si spostano. Ogni cambio di rete è una finestra di vulnerabilità.
- **Risorse limitate**: sensori IoT, smartwatch, embedded device hanno CPU e memoria ridotte. Difficile implementare crittografia pesante $\Rightarrow$ più vulnerabili a DoS e malware.
- **Accessibilità fisica**: un sensore incustodito in un campo agricolo o un magazzino può essere manomesso o ascoltato senza essere notato.

7.2 Tassonomia delle minacce

Prima di entrare nel tecnico, è utile una mappa delle principali categorie di minaccia.

Definition 7.2.1: Rogue Access Point

Un attaccante configura un dispositivo wireless affinché si comporti come un AP legittimo. Gli utenti si connettono credendo di essere sulla rete autentica, esponendo credenziali e traffico.

Definition 7.2.2: Reti ad hoc

Reti peer-to-peer tra dispositivi wireless *senza AP centrale*. La mancanza di un punto di controllo rende ogni nodo costretto a fidarsi direttamente degli altri.

Definition 7.2.3: MAC Spoofing

Ogni scheda di rete ha un indirizzo MAC a 48 bit. In teoria univoco e fisso, in pratica modificabile via software in pochi secondi. Basta sniffare il traffico, individuare un MAC autorizzato e impersonarlo.

Definition 7.2.4: Man-in-the-Middle (MitM)

L'attaccante si posiziona tra client e AP. Il client crede di parlare con l'AP, l'AP crede di parlare col client: in realtà tutto passa per l'attaccante.

Definition 7.2.5: Denial of Service (DoS)

Saturazione del canale o delle risorse dell'AP con messaggi continui, impedendo il funzionamento per gli utenti legittimi. Il mezzo radio è particolarmente vulnerabile dato che è fisicamente condiviso.

Definition 7.2.6: Network Injection

Iniezione di messaggi di protocollo fasulli (routing, gestione) verso AP esposti, con l'obiettivo di degradare le prestazioni o reindirizzare il traffico.

7.2.1 Attacchi passivi vs attivi

Definition 7.2.7: Attacco passivo

L'attaccante ascolta e basta, senza alterare il flusso. Esempi: *eavesdropping* (intercettazione) e *traffic analysis* (analisi dei metadati anche senza decifrare). Difficili da rilevare perché non lasciano tracce.

Definition 7.2.8: Attacco attivo

L'attaccante modifica, inietta o blocca il traffico. Rientrano in questa categoria: *masquerade*, *data modification*, *denial of service*, *replay attack*.

7.3 Attacchi al livello radio

7.3.1 Jamming

Il **jamming** è la forma più elementare di DoS wireless: trasmettere rumore o un segnale continuo sulla stessa frequenza della rete bersaglio.

Example 7.3.1 (OOK (On-Off Keying))

Nella modulazione OOK il bit 1 corrisponde alla portante accesa, il bit 0 alla portante spenta. Se l'attaccante tiene la portante **sempre accesa** (trasmette sempre "1111...") il ricevitore non distingue più i dati legittimi dal rumore. È come urlare di continuo mentre qualcuno cerca di sentire un sussurro.

Note:

Questo attacco richiede **molta energia**, proporzionale alla banda da disturbare.

7.3.2 Frequency Hopping Spread Spectrum (FHSS)

Per rendere il jamming molto più costoso si usa banda larga: invece di una singola frequenza portante, se ne usano **molte**, saltando da una all'altra secondo una sequenza pseudo-casuale (PRNG) concordata a priori tra Tx e Rx.

Proposition 7.3.1 Robustezza di FHSS

Un jammer che vuole disturbare la comunicazione deve disturbare **tutte** le frequenze contemporaneamente, richiedendo un'energia enormemente maggiore. In caso di collisione/jamming su una singola portante si perde solo la porzione di informazione trasmessa su quella frequenza.

Vantaggi:

- Migliora la sicurezza
- Riduce le interferenze
- Connessioni stabili
- Usato in Bluetooth, IoT, applicazioni militari

Note:

Conoscere il seed del PRNG di FHSS equivale a poter intercettare tutta la comunicazione: la sicurezza dipende interamente dalla segretezza del seed.

7.3.3 Eavesdropping / Sniffing

In un ambiente radio aperto, fare eavesdropping è semplice conoscendo tre cose:

- la **frequenza** (o la sequenza di frequenze nel caso FHSS)
- la **modulazione** usata
- il **protocollo**

Con queste info basta un ricevitore SDR (Software-Defined Radio) nel raggio del segnale. Nessun accesso fisico richiesto.

7.3.4 Replay Attack

C'è una categoria di attacchi che non richiede nemmeno di conoscere modulazione o protocollo: il **replay attack**.

Definition 7.3.1: Replay Attack

Catturare una porzione di spettro radio in un momento e ritrasmetterla identica in seguito. Non serve capire cosa c'è dentro: basta registrare e riprodurre.

Note:

Il canale radio NON è protetto da questo attacco dai meccanismi di confidenzialità o integrità! Se reinoltro un messaggio cifrato e integro, viene considerato valido.

Example 7.3.2 (Cancelli automatici)

I cancelli controllati via radiocomando usano spesso OOK a 433 MHz. Un attaccante con un RTL-SDR (ricevitore da pochi euro) cattura il treno di impulsi mentre apri il cancello, poi lo riproduce per aprirlo lui stesso, senza mai capire cosa significhi il segnale.

7.3.5 Rolling Code

Per difendersi dal replay naive, i sistemi più moderni usano il **Rolling Code**.

Definition 7.3.2: Rolling Code

Trasmettitore e ricevitore condividono un PRNG. Ogni pressione del telecomando invia il codice corrente, poi il trasmettitore avanza al codice successivo. Il ricevitore accetta solo se il codice è consistente con la sua posizione nel PRNG, e avanza anche lui.

Problema del disallineamento : se il trasmettitore invia ma il ricevitore non riceve (fuori portata), i due PRNG si disallineano. Soluzione: il ricevitore accetta codici in una **finestra** di N valori successivi a quello atteso (tipicamente ~ 100), poi sposta la finestra.

Question 1: Il Rolling Code risolve il replay?

No! Lo rende solo meno efficiente. Un attaccante può catturare vari codici prima che vengano usati dalla vittima, usarli in sequenza, poi ricatturarne altri. Per protezione robusta serve un **canale di ritorno**.

7.3.6 Challenge and Response

Il limite dei metodi precedenti è la **mancanza di feedback**: cancelli e telecomandi sono solo trasmettitori. Con **transceiver** (Tx + Rx) si può fare Challenge and Response.

Definition 7.3.3: Challenge-Response

1. Il client (autenticando) invia una richiesta di accesso.
2. Il server (autenticatore) genera un numero casuale fresco, il **nonce**, e lo invia al client.
3. Il client **cifra** il nonce con la chiave condivisa e risponde.
4. Il server decifra e verifica la corrispondenza.

Ogni sessione usa un nonce diverso, quindi un messaggio catturato non può essere riusato: il replay è neutralizzato strutturalmente.

7.3.7 Cifratura radio

Oltre che per autenticazione, la cifratura serve per **confidenzialità** e **integrità** dei dati in transito. Nel mondo radio si usano:

- **Cifrari a blocco**: es. AES (standard de facto oggi)
- **Cifrari a flusso**: es. **Kasumi / A5/3**, usato in 3G/4G GSM

Note:

Nel 2010 è stato pubblicato il "*sandwich attack*" (una variante di related-key attack) che ha rotto Kasumi permettendo di ricavare la chiave completa. Anche gli standard largamente adottati possono essere vulnerabili.

7.4 RFID

Definition 7.4.1: RFID (Radio Frequency IDentification)

Sistema di identificazione basato su campi elettromagnetici. Composto da:

- **Tag**: transponder radio miniaturizzato attaccato all'oggetto. Eccitato dal campo del lettore, trasmette ID e dati.
- **Lettore**: genera il campo e riceve la risposta.
- **Sistema informatico**: gestisce i dati ricevuti.

I tag possono essere **passivi** (alimentati dal campo del lettore, niente batteria) o **attivi** (con batteria, raggio maggiore). Usato ovunque: badge aziendali, carte contactless, antitaccheggio, passaporti biometrici.

Example 7.4.1 (Badge Unibo)

Usano la tecnologia **EM410X**: ogni badge ha un ID univoco, associato nel DB dell'università all'account.

Il varco consulta il DB per verificare l'autorizzazione.

7.4.1 Cloning dei tag RFID

I badge EM410X sono venduti con ID fisso e non modificabile. **Come aggirare la protezione?**

Note:

Basta comprare badge *scrivibili* compatibili con lo stesso standard, leggere l'ID del badge vittima con un lettore RFID, e scriverlo sul badge vuoto. Clone funzionante. Rasoio di Occam / percorso di minima resistenza: non si rompe la crittografia, ci si procura lo stesso strumento.

7.4.2 Spoofing RFID

Se non ci sono badge scrivibili ma il sistema è privo di crittografia, si usa un **transceiver RFID programmabile** (es. `Proxmark`) per emulare il segnale del tag legittimo:

1. L'attacker intercetta i segnali emessi dai tag RFID con un lettore.
2. L'attacker usa il dispositivo specializzato per **imitare quei segnali**.

La difesa fondamentale contro questi attacchi è l'uso di protocolli crittografici nell'RFID (challenge-response con chiavi vere). Tecnologie come **MIFARE DESFire** sono molto più resistenti di EM410X.

7.5 IEEE 802.11 — Wi-Fi

Lo standard **IEEE 802.11**, comunemente noto come **Wi-Fi**, è la tecnologia wireless più pervasiva nel mondo informatico. Crea **Wireless LAN (WLAN)** interoperabili con le reti Ethernet (IEEE 802.3).
Esistono diversi sotto-standard:

- **IEEE 802.11g**: reti a 2.4 GHz
- **IEEE 802.11ac/ax**: reti a 5 GHz

7.5.1 Layer fisico

A livello fisico **802.11 non prevede protezioni**. Le onde radio si propagano in tutte le direzioni attraverso i muri. L'unica "protezione" fisica possibile sono materiali schermanti (vernici tipo **YSHIELD**, gabbie di Faraday) che attenuano la propagazione fuori dall'ambiente controllato.

7.5.2 Accesso al canale: CSMA/CA

Le reti 802.11 usano un canale condiviso $\Rightarrow$ gestiscono le collisioni con **CSMA/CA**.

Definition 7.5.1: CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)

Prima di trasmettere un nodo "ascolta" il canale:

- Se occupato, aspetta.
- Se libero, aspetta comunque un tempo casuale (**backoff**) prima di trasmettere.

Questo riduce la probabilità che due nodi trasmettano nello stesso istante.

Problema del Terminale Nascosto

Non tutti i nodi si "vedono" a vicenda: l'AP di solito ha visibilità di tutta la rete, ma i singoli nodi no.

Example 7.5.1 (Hidden terminal)
Tre nodi: A, B (AP), C. Sia A che C sono nel raggio di B ma non nel raggio l'uno dell'altro.

$$[A] — (AP : B) — [C]$$

C non sa quando A sta trasmettendo $\Rightarrow$ può trasmettere credendo il canale libero $\Rightarrow$ collisione sull'AP. È una criticità strutturale delle reti wireless.

Inter-Frame Space (IFS)

Dopo una collisione, tutti i nodi aspettano un **Inter-Frame Space (IFS)** prima di ritrasmettere. Il tempo totale è:

$$\text{DIFS} + \text{backoff}$$

dove il backoff è estratto uniformemente in $[1, CW]$ con CW = Contention Window. Gli IFS hanno diverse priorità (SIFS, DIFS, EIFS) per diversi tipi di frame:

- Spazi attendibili solo dall'access point
- Spazi dedicati ai client
- ecc.

Appropriazione del canale (DoS su CSMA/CA)

Question 2: Cosa succede se qualcuno non rispetta l'IFS dopo una collisione?

Parla sempre e solo lui! Appropriazione totale della rete.

Question 3: E se due terminali non rispettano gli IFS?

Nessuno può più parlare! DoS totale.

7.6 Meccanismi di "sicurezza" deboli

7.6.1 SSID Hiding — Security by Obscurity

Definition 7.6.1: SSID Hiding

Disabilitare il broadcast del SSID del router: il nome della rete non appare nella lista delle reti disponibili. Per accedervi bisogna conoscerne il nome.

Il problema è fondamentale: quando un dispositivo che conosce già quella rete si riavvicina, manda **probe request in broadcast** chiedendo "C'è la rete X?" **con il nome in chiaro**. Chiunque abbia la scheda Wi-Fi in monitor mode può vedere questi probe e ricavare il nome.

Note:
Classico esempio di **security by obscurity**: una misura che dà una falsa sensazione di sicurezza senza fornire protezione reale.

7.6.2 MAC Address Whitelisting

Un altro "meccanismo" comune: l'AP accetta connessioni solo da MAC specifici.
Il problema è che l'indirizzo MAC viene trasmesso **in chiaro** in ogni frame 802.11. Un attaccante in ascolto vede i MAC autorizzati e con un semplice comando (`macchanger` su Linux) cambia il proprio MAC:

```
[root@snark ~]# macchanger -m 11:22:33:44:55:66 virbr0
Current MAC:    52:54:00:b7:9a:84 (unknown)
Permanent MAC:  00:00:00:00:00:00 (XEROX CORPORATION)
```

Ancora security by obscurity.

7.6.3 Management Frames in chiaro

L'autenticazione e la confidenzialità sono cruciali: in una rete **open** tutto il traffico viaggia in chiaro nell'etere, non serve nemmeno un MitM per sniffare.
Tuttavia, anche in reti cifrate, 802.11 prevede che alcuni **management frames** viaggino **in chiaro** e senza autenticazione/anti-replay. Tra questi: **deauthentication** e **disassociation**.
Le fasi dell'associazione 802.11 sono:

1. **Authentication phase** (Request/Response)
2. **Association phase** (Request/Response)
3. **Normal communication**

Deauthentication / Dissociation Attack

L'attaccante può inviare un messaggio di deauthentication falsificato (con MAC sorgente dell'AP legittimo, non verificato) a un client. Il client si disconnette immediatamente $\Rightarrow$ DoS istantaneo e silenzioso. Viene anche usato come primo passo di attacchi più complessi (es. forzare la riconnessione per catturare l'handshake WPA).

Note:

Soluzione: standard **IEEE 802.11w-2009**, noto come **Protected Management Frames (PMF)**. Obbligatorio in WPA3.

7.7 WEP — Wired Equivalent Privacy

Prima forma di autenticazione e confidenzialità in 802.11. Il nome era ambizioso: "privacy equivalente al cablato". La realtà è che **WEP è fondamentalmente rotto**, non va usato mai. Offre due modalità.

7.7.1 WEP — Shared Key

Tutti i dispositivi della rete condividono una chiave segreta. **Four-step challenge-response handshake**:

$$\begin{array}{ccc} \text{Client} & \xrightarrow{\text{Auth request}} & \text{AP} \\ \text{Client} & \xleftarrow{n=\text{challenge}} & \text{AP} \\ \text{Client} & \xrightarrow{RC4_k(n)} & \text{AP} \\ \text{Client} & \xleftarrow{\text{Accept/Reject}} & \text{AP} \end{array}$$

Scopo: il client dimostra di possedere la chiave.

Note:

Dannoso! Un attaccante che cattura lo scambio conosce sia il plaintext n (inviato in chiaro) sia il ciphertext $RC4(k, n)$. **Known Plaintext Attack (KPA)**: con abbastanza osservazioni ricava il keystream e quindi la chiave. Paradossalmente Shared Key è **meno sicura** dell'Open System.

7.7.2 WEP — Open System

Non c'è verifica con challenge. Chi si connette deve già conoscere il segreto comune, altrimenti non può decifrare i pacchetti dell'AP (e i suoi verranno scartati). L'autenticazione è implicita nella capacità di cifrare correttamente.
Struttura del pacchetto:

$$| \text{ IV } | \text{ Key ID } | \underbrace{\text{ Data } | \text{ ICV }}_{\text{Encrypted}} |$$

WEP con RC4

Definition 7.7.1: RC4 in WEP

Stream cipher basato sulla chiave k e sul **Initialization Vector** v (a 24 bit), genera un keystream $RC4(v, k)$.

Per inviare il messaggio M da Alice a Bob:

1. Calcola checksum di integrità a 32 bit (CRC32): $c(M)$
2. Plaintext: $P = \{M, c(M)\}$
3. Cifra P: $C = P \oplus RC4(v, k)$
4. Trasmette $C' = v \parallel C$

Per decifrare il processo è invertito.

7.7.3 WEP — IV Collision

Theorem 7.7.1 Birthday Paradox sugli IV

Con soli 24 bit l'IV ha $2^{24} \approx 16$ milioni di valori possibili. Dopo $\sim 30\,000$ pacchetti la probabilità di collisione supera il 50%:

$$P_{\text{collision}} \approx 1 - e^{-\frac{30000^2}{2 \cdot 2^{24}}} \approx 1$$

Note:

Il birthday paradox dice che in un gruppo di 23 persone scelte a caso, almeno due condividono il compleanno con probabilità $\geqslant 1/2$.

Se due frame usano lo stesso IV, usano lo stesso keystream: lo XOR dei ciphertext è uguale allo XOR dei plaintext. Da lì si fanno **attacchi statistici** per ricavare contenuti e, con abbastanza dati, la chiave. Catturando pacchetti con IV noto si fa inoltre **replay** ricircolando pacchetti vecchi.

Note:

Tool come `aircrack-ng` rompono WEP in pochi minuti su una rete trafficata.

7.8 IEEE 802.11i e WPA/WPA2

Per ovviare ai problemi di WEP è stato sviluppato **IEEE 802.11i**, implementato nella famiglia **Wi-Fi Protected Access (WPA)**, arrivata alla versione 3.

7.8.1 Modi di utilizzo

- **PSK (Pre-Shared Key, 256 bit)**: reti domestiche. La passphrase viene derivata in chiave a 256 bit tramite PBKDF2.
- **Enterprise**: autenticazione individuale tramite server RADIUS e protocollo EAP. Tipica di reti universitarie (ALMAWIFI, eduroam) e aziendali.
- **WPS**: sistema di autenticazione facilitato.

7.8.2 Schemi di protezione (802.11i)

IEEE 802.11i definisce due schemi per proteggere le **MPDU (MAC Protocol Data Unit)**:

- **TKIP (Temporal Key Integrity Protocol)**: compatibile con hardware WEP, oggi **deprecato**
- **CCMP (Counter Mode-CBC MAC Protocol)**: basato su **AES**, standard attuale

Formato MPDU:

$$\underbrace{|\ \text{MAC Ctrl}\ |\ \text{Dest}\ |\ \text{Source}\ |}_{\text{MAC header}}\ |\ \text{MSDU}\ |\ \underbrace{\text{CRC}}_{\text{trailer}}\ |$$

7.8.3 TKIP

Progettato per richiedere solo **modifiche software** ai dispositivi WEP. Fornisce:

Definition 7.8.1: Integrità in TKIP — MIC

Aggiunge un **Message Integrity Code (MIC)** a 64 bit al frame MAC dopo il campo Dati. Il MIC è generato dall'algoritmo **Michael** a partire da:

- MAC di origine e destinazione
- campo Dati
- materiale della chiave

Definition 7.8.2: Confidenzialità in TKIP

Cifra MPDU + MIC usando **RC4**.

Temporal Key (TK) di 256 bit

- 2×64 bit $\Rightarrow$ usate con Michael per il MIC:
  - Una chiave per STA $\rightarrow$ AP
  - Un'altra per AP $\rightarrow$ STA
- Restanti 128 bit $\Rightarrow$ troncati per generare la chiave RC4 di cifratura

TKIP Sequence Counter (TSC)

Definition 7.8.3: TSC

Contatore di sequenza assegnato a ciascun frame. Due scopi:

- Incluso in ogni MPDU e protetto dal MIC $\Rightarrow$ protegge dai **replay attack**.
- Combinato con la TK per produrre una **chiave dinamica** che cambia a ogni MPDU $\Rightarrow$ rende molto più difficile la crittoanalisi.

7.8.4 CCMP — Counter Mode-CBC MAC Protocol

Destinato ai dispositivi 802.11 recenti con hardware dedicato. Fornisce:

- **Integrità**: MAC basato su CBC (Cipher Block Chaining) $\Rightarrow$ **CBC-MAC**
- **Confidenzialità**: AES in modalità **CTR (Counter)**

Note:

Dettagli:

- **Stessa chiave AES a 128 bit** sia per integrità che per confidenzialità.
- **Packet counter a 48 bit** come nonce $\Rightarrow$ previene replay (2^{48} valori, riutilizzo praticamente impossibile).

7.9 Attacchi a WPA e WPS

7.9.1 Attacchi a WPA-PSK

Anche WPA2 con CCMP non è immune se la passphrase è debole. Durante il **4-way handshake** di autenticazione vengono scambiati messaggi che permettono ad un attaccante in ascolto di catturare materiale crittografico sufficiente a tentare:

- **Attacchi bruteforce**
- **Attacchi a dizionario** (offline)

Si prova ogni passphrase candidata, si deriva la chiave con PBKDF2, si confronta con l'handshake catturato. Con GPU moderne e dizionari buoni, le password deboli cadono in minuti.

7.9.2 WPS (Wi-Fi Protected Setup)

WPS semplifica la connessione senza scambio di passphrase (una sorta di prova di accesso fisico). Metodologie:

- **Tasto fisico** (pressione di un pulsante)
- **Codice PIN a 8 cifre**

Attacchi a WPS

Tasto fisico: aggirabile con Rogue AP in posizione tattica.
PIN: problema molto più preoccupante.

- Il PIN è a 7 cifre effettive (l'ottava è un checksum) $\Rightarrow 10^7 = 10\,000\,000$ tentativi teorici.
- L'attaccante recupera il PIN in poche ore con brute force e da lì recupera la **PSK WPA/WPA2**.

Theorem 7.9.1 Errore di design di WPS

Il PIN viene controllato in **due blocchi**:

- prima le prime 4 cifre
- poi le altre 3 (solo se le prime 4 sono corrette)

Quindi i tentativi reali sono

$$10^4 + 10^3 = 11\,000 \approx 20 \text{ ore di brute force}$$

non 10^7.

PixieDust

L'implementazione WPS su molti dispositivi embedded (TP-Link, D-Link, Netgear) usava un **generatore di nonce prevedibile**. L'attacco **PixieDust** riduce il brute force a **pochi minuti** catturando e analizzando la comunicazione.

Note:

Tool: `pixiewps` su Kali Linux (https://www.kali.org/tools/pixiewps/).

7.9.3 Rogue AP

Il settaggio del **nome della rete (SSID) non è crittografato**: solo l'accesso ad essa lo è. Questo permette attacchi **Rogue Access Point** con spoofing del nome: i client tentano l'accesso con la loro password e possono esporre handshake o credenziali all'attaccante.

Note:

Contromisura: WPA Enterprise con **certificati server** e autenticazione reciproca. Il client verifica l'identità dell'AP tramite certificato ⇒ un AP fasullo non passa la verifica.

7.10 KRACK e WPA3

7.10.1 KRACK — Key Reinstallation AttaCK

Attacco del 2017, uno dei più eleganti e importanti degli ultimi anni. Sfrutta una vulnerabilità nel **4-way handshake di WPA2**, non nella crittografia in sé ma nel protocollo di gestione delle chiavi.

Definition 7.10.1: KRACK

Nel 4-way handshake viene negoziata una chiave di sessione e un nonce. Se il client non invia un ACK del terzo messaggio, il server ritrasmette. L'attaccante manipola questo meccanismo per far sì che il client **reinstalli la stessa chiave**, reimpostando il nonce al valore iniziale.

Proposition 7.10.1 Perché è grave

Se il nonce viene riusato con la stessa chiave, si ottiene lo **stesso keystream** per messaggi diversi. Come in WEP, questo rompe la confidenzialità. In modalità CTR (CCMP/AES) il riuso del nonce permette di decifrare e in alcuni casi forgiare pacchetti.

Note:

Da https://www.krackattacks.com/: "*In a key reinstallation attack, the adversary tricks a victim into reinstalling an already-in-use key. [...] When the victim reinstalls the key, associated parameters such as the incremental transmit packet number (i.e. nonce) and receive packet number (i.e. replay counter) are reset to their initial value. Essentially, to guarantee security, a key should only be installed and used once.*"

KRACK è stato il catalizzatore dell'adozione di **WPA3**.

7.10.2 WPA3

WPA3 (2018) risolve le principali debolezze di WPA2:

Definition 7.10.2: SAE (Simultaneous Authentication of Equals)

Sostituisce il PSK nelle reti domestiche. Variante del **Dragonfly Key Exchange** (RFC 7664), basato su Diffie-Hellman. Offre **forward secrecy**: anche se un attaccante registra traffico cifrato oggi e in futuro scopre la passphrase, non può decifrare il traffico passato, perché le chiavi di sessione dipendono dai nonce freschi dell'handshake.

Vantaggi di WPA3:

- **SAE** ⇒ attacchi a dizionario offline molto più difficili
- **Forward secrecy** ⇒ traffico passato resta sicuro anche se la password viene scoperta dopo
- **PMF obbligatori** ⇒ l'attacco di deauthentication non funziona più
- **Resistenza al KRACK**: il protocollo di handshake è stato riprogettato

Lato negativo: richiede hardware recente, non sempre retrocompatibile con dispositivi vecchi.

7.11 Riepilogo e concetti chiave

7.11.1 Evoluzione della sicurezza Wi-Fi

Standard	Cifratura	Integrità	Auth	Problemi
Open	—	—	—	Tutto in chiaro
WEP Shared	RC4	CRC32	Challenge	KPA, IV collision, replay
WEP Open	RC4	CRC32	PSK	IV collision (birthday)
WPA (TKIP)	RC4 + TSC	Michael (MIC)	PSK / Enterprise	RC4 obsoleto, deprecato
WPA2 (CCMP)	AES-CTR	CBC-MAC	PSK / Ent. / WPS	KRACK, PSK brute, WPS
WPA3 (SAE)	AES-GCMP	—	Dragonfly/SAE	Richiede HW recente

7.11.2 Concetti chiave

1. **Il canale radio è intrinsecamente insicuro**: confidenzialità e integrità non bastano, serve **anti-replay** (nonce, sequence counter).
2. **Security by obscurity non funziona**: SSID hiding e MAC whitelist sono aggirabili con sniffing.
3. **PRNG prevedibili distruggono la sicurezza**: es. PixieDust su WPS.
4. **Riuso di chiavi/nonce è fatale**: IV collision in WEP, KRACK in WPA2. Una chiave va installata e usata **una volta sola**.
5. **Jamming è sempre possibile a livello fisico**: FHSS mitiga ma non elimina.
6. **Anche i management frames vanno protetti**: IEEE 802.11w (PMF) risolve i deauth attack.
7. **Autenticazione reciproca**: il client deve verificare anche l'identità dell'AP (certificati) $\Rightarrow$ blocca i Rogue AP.
8. **Forward secrecy**: compromissione futura della password non rende decifrabili comunicazioni passate.
9. **Ogni evoluzione dello standard è stata guidata da attacchi reali**: WEP $\rightarrow$ WPA $\rightarrow$ WPA2 $\rightarrow$ WPA3.

7.11.3 Strumenti e riferimenti menzionati

- `Wireshark` — sniffing di traffico
- `macchanger` — spoofing MAC
- `Proxmark` — spoofing RFID
- `pixiewps` (Kali Linux) — attacco PixieDust a WPS
- `aircrack-ng` — rottura di WEP
- KRACK attacks — https://www.krackattacks.com/
- RFC 7664 — Dragonfly Key Exchange
- IEEE 802.11w-2009 — Protected Management Frames